

Name: Draven Chen

Date: 10/27/2003

Final IDE Program Join Link: <https://onlinegdb.com/NrNX7BqZO3>

PART 1: Defining Your Problem

Task

State the problem you are planning to solve.

Requirements

Describe the problem you are trying to solve.

Describe any input data you expect to use.

Describe what the program will do to solve the problem.

Describe any outputs or results the program will provide.

Inspiration

Is your problem clearly defined?

Why do you want to solve this particular problem?

What source(s) of data do you believe you will need? Will the user need to supply that data, or will you get it from an external file or another source?

Will you need to interact with the user throughout the program? Will users continually need to enter data in and see something to continue?

What are your expected results or what will be the end product? What will you need to tell a user of your program when it is complete?

I am trying to solve the problem of creating a Java program which simulates gacha summoning occurring in common mobile and online games. These games enable players to expend resources to acquire random characters or items, with the outcomes being probabilistic. I am interested in creating my program because it is a great way of practicing a number of Java concepts together – specifically classes, arrays or ArrayLists, loops, conditionals, random numbers, user input, and file output. The name of the player and choices from the menu during the runtime of the program will be the data that will be inputted into my program. The user/player will choose if they want to perform one summon, ten summons, view collection, view statistics, view pity counter or exit program. The user will not input the rates as the program will use the built in rates that go

along with the rates that most games use. The likelihood of obtaining a particular trading card within my program's catalogue is 60% common, 25% uncommon, 10% rare, 4% epic and 1% legendary.

The program solves the problem with the help of random characters by summoning the random character and assigning the character based on the random number. To combat the luck factor when summoning a character, the program will implement a pity system. Specifically, if the player plays too many summons without being granted a Legendary character, the next qualifying summon is guaranteed to be Legendary. Within my program, the player will be guaranteed to get a Legendary result on the next summon if the player reaches 89 summons in a row with no Legendary. The program will maintain various variables to store the total number of summons, counts of each rarity, duplicates, unique characters collected, and the current pity counter. The program produces results for each summon, lists the characters summoned, shows summon stats by rarity, and provides the current pity progress. Finally, it saves to a text file a summary of the session of the player.

PART 2: Working Through Specific Examples

Task

Write down clear and specific steps to solve a simple version of your problem you identified in Part 1.

Requirements

Complete the three steps below for at least two distinct examples/scenarios.

State any necessary input data for your simplified problem.

Write clear and specific steps in English (not Java) detailing what the program will do to solve the problem.

Describe the specific result of your example/scenario.

Inspiration

Are there any steps that you don't fully understand? These are places to spend more time working out the details. Consider adding additional smaller steps in these spots. Remember that a computer program is very literal. Are there any steps that are unclear? Try giving the steps of your example/scenario to a friend or family member

to read through and ask you questions about parts they don't understand. Rewrite these parts as clearly as you can.

Are there interesting edge cases for your program? Try to start one of your examples/scenarios with input that matches this edge case. How does it change how your program might work?

Scenario 1: A player performs one summon and gets an Epic character

For this example, the player's name and their decision to perform one summon are the input data. The gamer enters the name "Alex" in the textbox and selects the single summon option on the menu. The player name is stored by the program which shows the menu. Next, the participant opts to conduct an invocation. Afterward, the program verifies the pity counter to determine whether the next summon is guaranteed to be Legendary or not. Given the low pity counter in this example, a random number is generated between 1 and 100. The random number generated is 97. Since 97 is in the range of 96-99, which is Epic range, it will determine that the summon result must be Epic. Subsequently, the scheme randomly selects an Epic character from the Epic character pool such as "Archmage". Afterward, the scheme displays the summoning result to the player, updates the total summon number, increases the Epic count by 1 and adds that new character to the player board.

The specific result of this scenario is that the player receives an Epic character named "Archmage," the Epic count increases by one, the total summons increase by one, and the pity counter increases because the player did not receive a Legendary character.

Scenario 2: A player triggers the pity system

In this instance, the player inputs the name "Sam" and chooses the lone summon option. The input data required for this example is the name of the player, the option for one summon and an existing pity counter of 89. The program first stores the players information. It then examines the pity counter before producing a random number. As the pity counter has already reached 89, the next summon is eligible for pity usage. Consequently, rather than assigning a random number for rarity selection, the script sets the rarity directly to Legendary. The program will then randomly select one character from the legendary pool. For example, "Phoenix Queen" was chosen. Subsequently, results are showcased by program. Pity was triggered, a pop-up message says. The script adds 1 to the Legendary counter and changes the total summons as well. If the character was not owned by the player, it also adds that to their collection. Ultimately, the program

resets the pity counter to 0. The specific outcome for this situation would be the player receiving a guaranteed Legendary, the pity counter resetting to 0, and the Legendary counter increasing by 1.

Scenario 3: A player performs ten summons and receives a mix of results

In this case, the player's name and the option of performing ten summons would be the needed input data. In one case, the player types the name Jordan and selects the ten summon option. The program proceeds to a 10 iterations loop which will summon 10 characters. Before every summon, the program would check the pity counter and if the pity will not activate then the program would generate a random number in between 1 to 100 and then use the number to determine the rarity. Once the rarity has been decided, the program will select from the appropriate rarity pool and then display the result. Once the summon is complete, the total summons, total collections, rarity counts and pity counter will be updated. The characters may be new or duplicate. At the end of the loop, the program shows the 10 summon results. After the results of the 10 summons occur, updated session overall summons collections and count will occur. In the scenario stated above, the player has received a total of ten characters. Another outcome is the addition of any new characters pulled in the batch to the player's collection. The player's statistics will vary depending on the rarities pulled from the batch. Ultimately, the player's pity counter will be either at zero.

PART 3: Generalizing Into Pseudocode

Task

Write out the general sequence your program will use, including all specific examples/scenarios you provided in Part 2.

Requirements

Write pseudocode for the program in English but refer to Java program elements where they are appropriate. The pseudocode should represent the full functionality of the program, not just a simplified version. Pseudocode is broken down enough that the details of the program are no longer in any paragraph form. One statement per line is ideal.

Help With Writing Pseudocode

Here are a few links that can help you write pseudocode with examples. Remember to check out part 3 of the Example Journal Template Submission if you have not already. Note: everyone will write pseudocode differently. There is no right or wrong way to write it, other than to make sure you write it clearly and in as much detail as you can so that it should be easy to convert to code later.

<https://www.geeksforgeeks.org/how-to-write-a-pseudo-code/>
<https://www.wikihow.com/Write-Pseudocode>

Inspiration

Do you see common program elements and patterns in your specific examples/scenarios in Part 2, like variables, conditionals, functions, loops, and classes? These should be part of your pseudocode for the general sequence as well. Are there places where the steps for your examples/scenarios in Part 2 diverged? These may be places where errors may occur later in the project. Make note of them. When you are finished with your pseudocode, does it make sense, even to a person that does not know Java? Aim for the clearest description of the steps, as this will make it easier to convert into program code later.

```
Create class CharacterCard
    Store character name
    Store character rarity
    Store star rating
    Store whether the card is new to the collection

Create constructor for CharacterCard
    Set name
    Set rarity
    Set star rating
    Set isNew to true

Create methods to return name, rarity, and star rating
Create method to change isNew value
Create method to display the card as stars + name + rarity

Create class GachaPool
    Create ArrayList for Common characters
    Create ArrayList for Uncommon characters
    Create ArrayList for Rare characters
    Create ArrayList for Epic characters
    Create ArrayList for Legendary characters
    Create Random object

Create constructor for GachaPool
```

```

Initialize all ArrayLists
Fill each rarity list with three character objects

Create method pullCharacter(rarity)
    If rarity is Common
        Randomly choose one character from Common list
    Else if rarity is Uncommon
        Randomly choose one character from Uncommon list
    Else if rarity is Rare
        Randomly choose one character from Rare list
    Else if rarity is Epic
        Randomly choose one character from Epic list
    Else
        Randomly choose one character from Legendary list
    Return a new CharacterCard object with the chosen values

Create class Player
    Store player name
    Store collection as an ArrayList of CharacterCard
    Store total summons
    Store pity counter
    Store Common count
    Store Uncommon count
    Store Rare count
    Store Epic count
    Store Legendary count

Create constructor for Player
    Set player name
    Initialize collection
    Set all counters to 0

Create method addToCollection(card)
    Loop through collection
        If a card with the same name already exists
            Set card isNew to false
            Return false
    Set card isNew to true
    Add card to collection
    Return true

Create method incrementRarityCount(rarity)
    Increase the correct rarity counter

Create getter and setter methods for player data

Create class SummonResult
    Store CharacterCard
    Store pityTriggered as true or false

```

```

Create constructor for SummonResult
    Set card
    Set pityTriggered

Create getter methods for card and pityTriggered

Create class SummonSession
    Store Player object
    Store GachaPool object
    Store list of summon results for this session
    Create Random object
    Set pity threshold rule so next summon is guaranteed Legendary after
89 non-Legendary summons

Create constructor for SummonSession
    Save player and pool
    Initialize session results list

Create method performSingleSummon
    Set pityTriggered to false

    If player pity counter is greater than or equal to 89
        Set rarity to Legendary
        Set pityTriggered to true
    Else
        Generate random number from 1 to 100
        If number is 1 to 60
            Set rarity to Common
        Else if number is 61 to 85
            Set rarity to Uncommon
        Else if number is 86 to 95
            Set rarity to Rare
        Else if number is 96 to 99
            Set rarity to Epic
        Else
            Set rarity to Legendary

    Pull a character from the correct rarity pool
    Add the card to the player collection
    Increase total summons by 1
    Increase the correct rarity counter

    If rarity is Legendary
        Reset pity counter to 0
    Else
        Increase pity counter by 1

    Create SummonResult object

```

```

    Add result to session results list
    Return SummonResult

Create method performTenSummon
    Create empty result list
    Repeat 10 times
        Call performSingleSummon
        Add result to result list
    Return result list

Create main program class GachaSimulator
    Create Scanner for keyboard input
    Create GachaPool object
    Ask user for player name
    Create Player object
    Set continuePlaying to true

    While continuePlaying is true
        Display menu
            1 = Single Summon
            2 = Ten Summon
            3 = View Collection
            4 = View Statistics
            5 = View Pity Counter
            6 = Exit and Save

        Read menu choice safely
        If input is not a number
            Show error message
            Ask again

        If choice is 1
            Create SummonSession
            Perform one summon
            Display result
            If pityTriggered is true
                Display pity message

        Else if choice is 2
            Create SummonSession
            Perform ten summons
            Display all ten results
            Mark any pity result

        Else if choice is 3
            Display all unique characters in collection
            Display total unique characters

        Else if choice is 4

```

```

        Display total summons
        Display counts for each rarity
        If total summons is greater than 0
            Display percentages for each rarity

    Else if choice is 5
        Display current pity counter
        Display how many summons remain until guaranteed Legendary

    Else if choice is 6
        Save player session summary to text file
        Display goodbye message
        End loop

    Else
        Display invalid menu choice message

Create method to write session log to a text file
    Build a formatted summary string
    Include player name, date, total summons, collection size, rarity
    totals, and pity counter
    Write string to text file using append mode
    If a file error happens
        Display an error message

```

PART 4: Testing Your Program

Task

While writing and testing your program code, describe your tests, record any errors, and state your approach to fixing the errors.

Requirements

For at least one of your test cases, describe how your choices for the test helped you understand whether the program was running correctly or not.

For each error that occurs while writing and testing your code:

Record the details of the error from your IDE. A screenshot or copy-and-paste of the text into the journal entry is acceptable.

Describe what you attempted in order to fix the error. Clearly identify which approach was the one that worked.

Inspiration

Have you tested edge cases and special cases for the inputs of your program code?
Often these unexpected values can cause errors in the operation of your program.
Have you tested opportunities for user error? If a user is asked to provide an input, what happens when they give the wrong type of input, like a letter instead of a number, or vice versa?
Did the outcome look the way you expected? Was it formatted correctly?
Does your output align with the solution to the problem you coded for?

I used several tests while writing my program because this project depends on both correct calculations and correct user interaction. I especially focused on the randomness system, the pity system, duplicate detection, input validation, and file output. These were the parts of the program most likely to fail because they involve loops, conditionals, and updating multiple values at once.

One of my most important test cases was checking the pity system at the exact boundary. I chose this test because boundary values often reveal off-by-one errors. I set the player's pity counter to 89 and then performed one summon. Since my program is designed so that the next summon after 89 non-Legendary pulls is guaranteed to be Legendary, I expected the result to be Legendary and I expected the pity counter to reset to 0. This test helped me confirm whether the pity system was working exactly where it should. After fixing the bug described below, the program behaved correctly and the pity summon displayed a Legendary result with the pity message.

Another test case involved invalid menu input. At first, when I entered letters instead of a number at the menu, the program crashed. The IDE showed this runtime error:

```
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:939)
    at java.base/java.util.Scanner.nextInt(Scanner.java:1594)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
    at GachaSimulator.main(GachaSimulator.java:214)
```

To fix this problem, I added a validation loop with a try-catch block around the menu input. I also used `input.nextLine()` inside the catch block so the program would clear the invalid value before asking again. This was the fix that worked because the program no longer crashed when I typed text, and it kept prompting the user until a valid number was entered.

I also had a compile-time error when I first wrote the input validation because I forgot to import `InputMismatchException`. The IDE showed this error message:

```
error: cannot find symbol
catch (InputMismatchException e)
    ^
symbol:   class InputMismatchException
location: class GachaSimulator
```

I fixed this by adding `import java.util.InputMismatchException;` at the top of the program. After that, the program compiled correctly and the input validation worked.

Another issue I found during testing involved duplicate detection. When I summoned the same character more than once, the collection size kept increasing even though duplicates were supposed to be marked instead of added again. This was a logic error rather than a compiler error. I discovered that I had compared character names incorrectly. After changing the comparison to use `.equals()` for the character names, duplicates were handled correctly. New characters were added to the collection, while repeated characters were shown as duplicates and did not increase the collection size.

I also tested file output because I wanted to make sure the session summary would be readable. At first, the information in the log file appeared crowded and difficult to read. I fixed that by adding clear labels and line breaks between sections of the summary. After the fix, the file correctly recorded the player name, date, summon totals, rarity counts, collection size, and pity counter.

I tested the file output and confirmed that each session appends to the log. The program currently does not read previous session data back in, so the collection and pity counter reset to zero on each new run.

Finally, I tested the ten summon option to make sure the loop ran exactly ten times and updated the statistics correctly after every summon. I compared the number of results displayed to the change in total summons and verified that they always matched. This test helped confirm that the loop, the result display, and the statistic counters were all staying synchronized.

PART 5: Commenting Your Program

Task

Submit your full program code, including thorough comments describing what each portion of the program should do when working correctly.

Requirements

The purpose of the program and each of its parts should be clear to a reader that does not know the Java programming language.

Inspiration

Is each section or sub-section of your code commented to describe what the code is doing?

Give your code with comments to a friend or family member to review. Add additional comments to spots that confuse them to make it clearer.

```
import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Random;
import java.util.Scanner;
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/*
 * CharacterCard represents one character that can be summoned.
 * Each character has a name, a rarity level, a star rating,
 * and a label showing whether the character is new or a duplicate.
 */
class CharacterCard {
    private String name;
    private String rarity;
    private int starRating;
    private boolean isNew;

    public CharacterCard(String name, String rarity, int starRating) {
        this.name = name;
        this.rarity = rarity;
    }
}
```

```

        this.starRating = starRating;
        this.isNew = true;
    }

    public String getName() {
        return name;
    }

    public String getRarity() {
        return rarity;
    }

    public int getStarRating() {
        return starRating;
    }

    public boolean isNew() {
        return isNew;
    }

    public void setIsNew(boolean value) {
        isNew = value;
    }

    /*
     * This method builds the visual display for a character result.
     * It shows stars based on the rarity and adds a label if the card is new.
     */
    @Override
    public String toString() {
        StringBuilder stars = new StringBuilder();

        for (int i = 0; i < starRating; i++) {
            stars.append("★");
        }

        String newTag;
        if (isNew) {
            newTag = " [NEW]";
        } else {
            newTag = " [DUPLICATE]";
        }

        return stars + " " + name + " (" + rarity + ")" + newTag;
    }
}

/*
 * GachaPool stores all available characters separated by rarity.
 * When the program decides the rarity of a summon, this class
 * chooses one random character from the correct pool.
 */
class GachaPool {
    private ArrayList<CharacterCard> commonPool;
    private ArrayList<CharacterCard> uncommonPool;

```

```

private ArrayList<CharacterCard> rarePool;
private ArrayList<CharacterCard> epicPool;
private ArrayList<CharacterCard> legendaryPool;
private Random random;

public GachaPool() {
    commonPool = new ArrayList<>();
    uncommonPool = new ArrayList<>();
    rarePool = new ArrayList<>();
    epicPool = new ArrayList<>();
    legendaryPool = new ArrayList<>();
    random = new Random();
    loadCharacters();
}

/*
 * This method fills each rarity pool with sample characters.
 * The program uses three characters per rarity level.
 */
private void loadCharacters() {
    commonPool.add(new CharacterCard("Squire", "Common", 1));
    commonPool.add(new CharacterCard("Apprentice", "Common", 1));
    commonPool.add(new CharacterCard("Villager", "Common", 1));

    uncommonPool.add(new CharacterCard("Knight", "Uncommon", 2));
    uncommonPool.add(new CharacterCard("Ranger", "Uncommon", 2));
    uncommonPool.add(new CharacterCard("Cleric", "Uncommon", 2));

    rarePool.add(new CharacterCard("Paladin", "Rare", 3));
    rarePool.add(new CharacterCard("Assassin", "Rare", 3));
    rarePool.add(new CharacterCard("Druid", "Rare", 3));

    epicPool.add(new CharacterCard("Archmage", "Epic", 4));
    epicPool.add(new CharacterCard("Shadow Lord", "Epic", 4));
    epicPool.add(new CharacterCard("Beastmaster", "Epic", 4));

    legendaryPool.add(new CharacterCard("Dragon Knight", "Legendary", 5));
    legendaryPool.add(new CharacterCard("Phoenix Queen", "Legendary", 5));
    legendaryPool.add(new CharacterCard("Void Walker", "Legendary", 5));
}

/*
 * This helper method creates a fresh copy of a selected card.
 * Returning a new object prevents one summon result from changing
 * the stored original card inside the pool.
 */
private CharacterCard copyRandomCard(ArrayList<CharacterCard> pool) {
    CharacterCard base = pool.get(random.nextInt(pool.size()));
    return new CharacterCard(base.getName(), base.getRarity(),
base.getStarRating());
}

/*
 * This method receives a rarity label and returns a random character
 * from the matching rarity pool.

```

```

    */
    public CharacterCard pullCharacter(String rarity) {
        switch (rarity) {
            case "Common":
                return copyRandomCard(commonPool);
            case "Uncommon":
                return copyRandomCard(uncommonPool);
            case "Rare":
                return copyRandomCard(rarePool);
            case "Epic":
                return copyRandomCard(epicPool);
            case "Legendary":
                return copyRandomCard(legendaryPool);
            default:
                return null;
        }
    }
}

/*
 * Player stores all information related to the user.
 * This includes the name, collection of unique characters,
 * summon totals, rarity totals, and pity progress.
 */
class Player {
    private String playerName;
    private ArrayList<CharacterCard> collection;
    private int totalSummons;
    private int pityCounter;
    private int commonCount;
    private int uncommonCount;
    private int rareCount;
    private int epicCount;
    private int legendaryCount;

    public Player(String playerName) {
        this.playerName = playerName;
        collection = new ArrayList<>();
        totalSummons = 0;
        pityCounter = 0;
        commonCount = 0;
        uncommonCount = 0;
        rareCount = 0;
        epicCount = 0;
        legendaryCount = 0;
    }

    public String getPlayerName() {
        return playerName;
    }

    public ArrayList<CharacterCard> getCollection() {
        return collection;
    }
}

```

```

public int getCollectionSize() {
    return collection.size();
}

public int getTotalSummons() {
    return totalSummons;
}

public void incrementTotalSummons() {
    totalSummons++;
}

public int getPityCounter() {
    return pityCounter;
}

public void setPityCounter(int pityCounter) {
    this.pityCounter = pityCounter;
}

public int getCommonCount() {
    return commonCount;
}

public int getUncommonCount() {
    return uncommonCount;
}

public int getRareCount() {
    return rareCount;
}

public int getEpicCount() {
    return epicCount;
}

public int getLegendaryCount() {
    return legendaryCount;
}

/*
 * This method updates the correct rarity counter after a summon.
 */
public void incrementRarityCount(String rarity) {
    switch (rarity) {
        case "Common":
            commonCount++;
            break;
        case "Uncommon":
            uncommonCount++;
            break;
        case "Rare":
            rareCount++;
            break;
        case "Epic":

```

```

        epicCount++;
        break;
    case "Legendary":
        legendaryCount++;
        break;
    }
}

/*
 * This method adds a character to the collection only if the player
 * does not already own that character. It also marks whether the result
 * should be shown as new or duplicate.
 */
public boolean addToCollection(CharacterCard card) {
    for (CharacterCard owned : collection) {
        if (owned.getName().equals(card.getName())) {
            card.setIsNew(false);
            return false;
        }
    }

    card.setIsNew(true);
    collection.add(card);
    return true;
}

/*
 * SummonResult stores the result of one summon.
 * It includes the card itself and whether pity was triggered.
 */
class SummonResult {
    private CharacterCard card;
    private boolean pityTriggered;

    public SummonResult(CharacterCard card, boolean pityTriggered) {
        this.card = card;
        this.pityTriggered = pityTriggered;
    }

    public CharacterCard getCard() {
        return card;
    }

    public boolean isPityTriggered() {
        return pityTriggered;
    }
}

/*
 * SummonSession handles the summoning logic.
 * It decides rarity, checks pity, updates statistics,
 * and returns the results to the main program.
 */
class SummonSession {

```

```

private Player player;
private GachaPool gachaPool;
private ArrayList<SummonResult> sessionResults;
private Random random;

/*
 * If the player reaches 89 consecutive non-Legendary pulls,
 * the next summon becomes a guaranteed Legendary.
 */
private final int PITY_READY_VALUE = 89;

public SummonSession(Player player, GachaPool gachaPool) {
    this.player = player;
    this.gachaPool = gachaPool;
    sessionResults = new ArrayList<>();
    random = new Random();
}

/*
 * This method performs one summon.
 * It checks pity first, then uses a random number if pity is not active.
 */
public SummonResult performSingleSummon() {
    String rarity;
    boolean pityTriggered = false;

    if (player.getPityCounter() >= PITY_READY_VALUE) {
        rarity = "Legendary";
        pityTriggered = true;
    } else {
        int roll = random.nextInt(100) + 1;

        if (roll <= 60) {
            rarity = "Common";
        } else if (roll <= 85) {
            rarity = "Uncommon";
        } else if (roll <= 95) {
            rarity = "Rare";
        } else if (roll <= 99) {
            rarity = "Epic";
        } else {
            rarity = "Legendary";
        }
    }

    CharacterCard card = gachaPool.pullCharacter(rarity);

    player.incrementTotalSummons();
    player.incrementRarityCount(rarity);
    player.addToCollection(card);

    if (rarity.equals("Legendary")) {
        player.setPityCounter(0);
    } else {
        player.setPityCounter(player.getPityCounter() + 1);
    }
}

```

```

    }

    SummonResult result = new SummonResult(card, pityTriggered);
    sessionResults.add(result);
    return result;
}

/*
 * This method performs ten summons in a row and stores each result.
 */
public ArrayList<SummonResult> performTenSummon() {
    ArrayList<SummonResult> batchResults = new ArrayList<>();

    for (int i = 0; i < 10; i++) {
        batchResults.add(performSingleSummon());
    }

    return batchResults;
}

public ArrayList<SummonResult> getSessionResults() {
    return sessionResults;
}
}

/*
 * GachaSimulator is the main class for the entire program.
 * It handles input, displays menus, calls summon methods,
 * shows statistics, and saves a session log when the player exits.
 */
public class GachaSimulator {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        GachaPool pool = new GachaPool();

        System.out.println("=====");
        System.out.println("  DRAVEN'S GACHA SUMMON SIMULATOR");
        System.out.println("=====");
        System.out.print("Enter your player name: ");
        String playerName = input.nextLine();

        Player player = new Player(playerName);
        boolean continuePlaying = true;

        while (continuePlaying) {
            displayMenu();
            int choice = readMenuChoice(input);

            switch (choice) {
                case 1:
                    singleSummon(player, pool);
                    break;
                case 2:
                    tenSummon(player, pool);

```

```

        break;
    case 3:
        showCollection(player);
        break;
    case 4:
        showStatistics(player);
        break;
    case 5:
        showPityCounter(player);
        break;
    case 6:
        saveSessionLog(player);
        System.out.println("Session saved. Goodbye, " +
player.getPlayerName() + "!");
        continuePlaying = false;
        break;
    default:
        System.out.println("Invalid choice. Please enter a number
from 1 to 6.");
    }
}

input.close();
}

/*
 * This method prints the main menu options.
 */
public static void displayMenu() {
    System.out.println();
    System.out.println("1. Single Summon");
    System.out.println("2. Ten Summon");
    System.out.println("3. View Collection");
    System.out.println("4. View Statistics");
    System.out.println("5. View Pity Counter");
    System.out.println("6. Exit and Save");
    System.out.print("Enter your choice: ");
}

/*
 * This method safely reads the player's menu choice.
 * It prevents the program from crashing if the user enters text.
 */
public static int readMenuChoice(Scanner input) {
    while (true) {
        try {
            int choice = input.nextInt();
            input.nextLine();

            if (choice >= 1 && choice <= 6) {
                return choice;
            } else {
                System.out.print("Please enter a number from 1 to 6: ");
            }
        } catch (InputMismatchException e) {

```

```

        System.out.print("Invalid input. Please enter a number from 1
to 6: ");
        input.nextLine();
    }
}

/*
 * This method performs one summon and prints the result.
 */
public static void singleSummon(Player player, GachaPool pool) {
    SummonSession session = new SummonSession(player, pool);
    SummonResult result = session.performSingleSummon();

    System.out.println();
    System.out.println("Single Summon Result:");
    System.out.println(result.getCard());

    if (result.isPityTriggered()) {
        System.out.println("PITY TRIGGERED! You received a guaranteed
Legendary.");
    }
}

/*
 * This method performs ten summons and prints all ten results.
 */
public static void tenSummon(Player player, GachaPool pool) {
    SummonSession session = new SummonSession(player, pool);
    ArrayList<SummonResult> results = session.performTenSummon();

    System.out.println();
    System.out.println("Ten Summon Results:");

    for (int i = 0; i < results.size(); i++) {
        System.out.println((i + 1) + ". " + results.get(i).getCard());

        if (results.get(i).isPityTriggered()) {
            System.out.println("    PITY TRIGGERED! Guaranteed Legendary.");
        }
    }
}

/*
 * This method shows all unique characters collected by the player.
 */
public static void showCollection(Player player) {
    System.out.println();
    System.out.println("Your Collection:");

    if (player.getCollection().isEmpty()) {
        System.out.println("You have not collected any characters yet.");
        return;
    }
}

```

```

        for (CharacterCard card : player.getCollection()) {
            System.out.println("- " + card.getName() + " (" + card.getRarity()
+ ")");
        }

        System.out.println("Total unique characters: " +
player.getCollectionSize() + " / 15");
    }

    /*
    * This method displays the player's total summons and rarity totals.
    * It also calculates percentages if the player has already summoned.
    */
    public static void showStatistics(Player player) {
        System.out.println();
        System.out.println("Summon Statistics:");
        System.out.println("Total summons: " + player.getTotalSummons());
        System.out.println("Common: " + player.getCommonCount());
        System.out.println("Uncommon: " + player.getUncommonCount());
        System.out.println("Rare: " + player.getRareCount());
        System.out.println("Epic: " + player.getEpicCount());
        System.out.println("Legendary: " + player.getLegendaryCount());

        if (player.getTotalSummons() > 0) {
            System.out.printf("Common Rate: %.1f%%\n",
                player.getCommonCount() * 100.0 /
player.getTotalSummons());
            System.out.printf("Uncommon Rate: %.1f%%\n",
                player.getUncommonCount() * 100.0 /
player.getTotalSummons());
            System.out.printf("Rare Rate: %.1f%%\n",
                player.getRareCount() * 100.0 / player.getTotalSummons());
            System.out.printf("Epic Rate: %.1f%%\n",
                player.getEpicCount() * 100.0 / player.getTotalSummons());
            System.out.printf("Legendary Rate: %.1f%%\n",
                player.getLegendaryCount() * 100.0 /
player.getTotalSummons());
        }
    }

    /*
    * This method displays the current pity progress.
    * The player is guaranteed a Legendary on the next summon
    * once the pity counter reaches 89.
    */
    public static void showPityCounter(Player player) {
        System.out.println();
        System.out.println("Pity Counter: " + player.getPityCounter());

        int remaining = 90 - player.getPityCounter();

        if (remaining < 1) {
            remaining = 1;
        }
    }

```

```

        System.out.println("Summons until guaranteed Legendary: " + remaining);
    }

    /*
     * This method saves a readable summary of the player's session to a text
    file.
     * The file is saved in append mode so multiple runs can be recorded.
     */
    public static void saveSessionLog(Player player) {
        File logFile = new File("gacha_session_log.txt");
        String lineBreak = System.lineSeparator();

        StringBuilder log = new StringBuilder();
        log.append("=====").append(lineBreak);
        log.append("Gacha Session Summary").append(lineBreak);
        log.append("Player:
    ").append(player.getPlayerName()).append(lineBreak);
        log.append("Date: ")

        .append(LocalDate.now().format(DateTimeFormatter.ofPattern("MM/dd/yyyy
    HH:mm:ss")))
        .append(lineBreak);
        log.append("Total Summons:
    ").append(player.getTotalSummons()).append(lineBreak);
        log.append("Collection Size:
    ").append(player.getCollectionSize()).append(" / 15").append(lineBreak);
        log.append("Common:
    ").append(player.getCommonCount()).append(lineBreak);
        log.append("Uncommon:
    ").append(player.getUncommonCount()).append(lineBreak);
        log.append("Rare: ").append(player.getRareCount()).append(lineBreak);
        log.append("Epic: ").append(player.getEpicCount()).append(lineBreak);
        log.append("Legendary:
    ").append(player.getLegendaryCount()).append(lineBreak);
        log.append("Current Pity Counter:
    ").append(player.getPityCounter()).append(lineBreak);

        log.append("=====").append(lineBreak).append(l
        ineBreak);

        try {
            Files.writeString(logFile.toPath(), log.toString(),
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
        } catch (IOException e) {
            System.out.println("Error saving session log: " + e.getMessage());
        }
    }
}

```

PART 6: Your Completed Program

Task

Provide the IDE link to your full program code.

Requirements

The program must work correctly with all the comments included in the program.

Inspiration

Check before submitting your Touchstone that your final version of the program is running successfully.

Final IDE Program Join Link: <https://onlinegdb.com/NrNX7BqZO3>

My completed Java program is a gacha summoning simulator that allows the user to perform single summons, perform ten summons, view collected characters, display summon statistics, check pity progress, and save a session log when exiting. The final version of the program includes comments throughout the code so that a reader who does not know Java can still understand the purpose of each class, method, and major section of the program. Before submitting this Touchstone, I will test the full program one final time in my IDE and confirm that the shared link opens the completed code correctly.